

Relatório Técnico: Migração de AI Studio (API Key) para Vertex AI (GCP)

1. Escopo e Objetivo

Este documento detalha o processo de migração de uma aplicação baseada em Python que utiliza modelos LLM do Google.

- **Cenário Atual:** Autenticação via Google AI Studio utilizando API Key.
- **Cenário Alvo:** Autenticação e inferência via Vertex AI no Google Cloud Platform (GCP).
- **Objetivo de Negócio:** Viabilizar o consumo de R\$ 1.724,76 em créditos disponíveis na conta do GCP.
- **Perfil da Carga de Trabalho:** Processamento de documentos longos (aprox. 110.000 tokens) divididos em blocos de 5.000 tokens para reescrita.

2. Pré-requisitos de Infraestrutura (GCP)

A transição exige a adequação da governança do projeto na nuvem, abandonando o modelo de chave estática em favor do IAM (Identity and Access Management).

2.1. Vínculo de Faturamento

O projeto no GCP deve estar vinculado à "Billing Account" que detém os créditos. Isso garante que cada requisição feita pela API seja debitada do saldo promocional antes de cobrar o cartão de crédito.

2.2. Ativação de Serviços

No console do GCP, é obrigatório ativar a **Vertex AI API**. Requisições feitas sem a ativação prévia resultarão em erros de permissão.

3. Arquitetura de Autenticação (Service Account)

Chaves de API representam um risco de segurança e não são suportadas pela Vertex AI para chamadas em produção. A autenticação ocorrerá via Service Account.

3.1. Criação e Permissionamento

Deve-se criar uma Service Account específica para a aplicação e conceder a ela a role de `roles/aiplatform.user` (Usuário da Vertex AI). Isso segue o princípio do privilégio mínimo.

3.2. Configuração no Ambiente de Execução

A credencial JSON gerada deve ser injetada no ambiente da aplicação através de variável de ambiente.

```
export GOOGLE_APPLICATION_CREDENTIALS="/caminho/seguro/credentials.json"
```

4. Refatoração do Código-Fonte (Python)

A lógica de fragmentação (110k para blocos de 5k) não muda. A alteração ocorre na SDK cliente.

Remoção e Instalação de Dependências:

```
pip uninstall google-generativeai
pip install google-cloud-aiplatform
```

Comparativo de Implementação: AI Studio vs Vertex AI

Google AI Studio (Descontinuar)	Vertex AI / GCP (Implementar)
<pre>import google.generativeai as genai genai.configure(api_key="CHAVE") model = genai.GenerativeModel('gemini-1.5-pro') resposta = model.generate_content("Reescreva: " + texto) print(resposta.text)</pre>	<pre>import vertexai from vertexai.generative_models import GenerativeModel # Autenticacao via env var vertexai.init(project="ID_DO_PROJETO", location="us-central1") model = GenerativeModel('gemini-1.5- pro') resposta = model.generate_content("Reescreva: " + texto) print(resposta.text)</pre>

5. Avaliação da Estratégia de Processamento (Chunking)

Manutenção da Arquitetura Atual: Embora os modelos Gemini 1.5 suportem mais de 1 milhão de tokens de entrada (o que permitiria enviar o documento inteiro de 110.000 tokens de uma vez), o limite máximo de **saída gerada (output)** é de 8.192 tokens.

Solicitar a reescrita integral do documento em um único prompt causaria um corte abrupto na resposta. A estratégia atual de enviar e receber blocos menores (5.000 tokens) mantém o aplicativo funcional e alinhado aos limites técnicos de saída do modelo.

6. Gestão de Custos (Consumo de R\$ 1.724,76)

O consumo dos créditos pode ser monitorado diariamente no console do GCP em **Faturamento > Relatórios**. Recomenda-se configurar alertas de orçamento (Budgets & Alerts) para enviar notificações automáticas via e-mail quando o consumo atingir 50%, 75% e 90% do saldo total disponível, evitando cobranças surpresas no cartão vinculado após o término da cota promocional.